CANNBot开发进阶：PyPTO 算子开发实操

作者：王政

时间：2026-04-15

目录

Part 1 背景介绍

Part 2 Agent 架构

Part 3 案例实操

PyPTO 简介

PyPTO（发音：pai p-t-o）是一款面向AI加速器的**高效编程框架**，采用创新的**PTO（Parallel Tensor/Tile Operation）编程范式**，以基于Tile的编程模型为核心，通过多层次IR系统将高层Tensor图逐步编译为硬件可执行代码。

481	447	v0.2.0
Stars	Forks	最新版本

核心特性

基于 Tensor/Tile 的编程模型
充分利用硬件并行计算能力与内存层次结构

多层级计算图转换
Tensor → Tile → Block → Execution 逐步编译优化

自动化代码生成
编译结果自动生成底层指令，部署到目标平台

Python 友好 API
直观的Tensor级别抽象，支持动态Shape

目标用户

- **算法开发者**
  Tensor层次编程，快速实现验证算法
- **性能优化专家**
  支持开箱优化和深度性能调优
- **系统开发者**
  框架对接、工具链开发与集成

已适配大模型

DeepSeek 系列

GLM 系列

https://gitcode.com/cann/skills

CANN

融合算子开发目标

CANNBot PyPTO 融合算子高效开发目标

降低开发周期

从编码、精度调试到性能调优，一周内全流程打通，提升迭代效率。

固化调优经验

Tiling 策略、流水配置、Loop 切分等关键调优环节形成标准化流程，不再依赖专家经验

降低上手难度

轻松调用 PyPTO 框架的功能与调优能力，降低使用门槛与学习成本

我们的解决方案

将专家经验固化为

7 个标准化 Skill

由

4 个专业 Agent

协同调度，构建端到端
自动化开发流程

https://gitcode.com/cann/skills

CANN

目录

Part 1 背景介绍

Part 2 Agent 架构

Part 3 案例实操

CANN

CANNBot PyPTO 三层架构

T TEAMS(应用编排层)
应用编排

pypto-op-orchestrator
PyPTO 算子开发流程

ops-direct-invoke
Ascend C 算子直调开发流程

A AGENTS(角色执行层)
角色执行

analyst
分析设计

developer
开发调试

perf-tuner
性能调优

S SKILLS(知识能力层)
知识能力

intent-understand	api-explore	golden-generate	op-design	op-develop	precision-debug	op-perf-tune
需求理解	API探索	Golden生成	方案设计	代码开发	精度调试	性能调优

编排层：Team编排Agents，驱动7阶段标准化开发流程

执行层：Agents绑定Skills，执行分析、开发、调优任务

能力层：Skills提供7个标准化能力，支撑端到端自动化

https://gitcode.com/cann/skills

CANN

4 个专业 Agent 协同工作

"Agent 编排 + Skill 执行" 分层架构 — 各司其职，协同完成全流程

编排师

pypto-op-orchestrator

全流程唯一 Owner

- 7 阶段状态机管理
- 工件门禁与重试控制
- 断点续跑与失败恢复
- 负责执行 Stage 1~2 和调度 3 个 SubAgent

分析师

pypto-op-analyst

分析型产物专家

- 生成 PyTorch 参考实现
- 输出完整设计文档
- 在隔离上下文中执行
- 负责执行 Stage 3~4

开发者

pypto-op-developer

实现与修复执行器

- 自动生成 Kernel 源码
- 三态判定与失败分类
- 代码检查与精度修复
- 负责执行 Stage 5~6

调优师

pypto-op-perf-tuner

性能迭代优化器

- 性能基线记录与分析
- Tiling / Loop / 流水搜索
- 精度优先的采纳策略
- 负责执行 Stage 7

https://gitcode.com/cann/skills

CANN

7 阶段标准化开发流程

1 需求理解	2 API 探索	3 Golden 生成	4 方案设计	5 代码实现	6 精度调试	7 性能调优

1 需求理解

解析自然语言描述
提取 shape / dtype 约束

→ SPEC.md

2 API 探索

检索 PyPTO API 库
验证原语与组合可行性

→ API_REPORT.md

3 Golden 生成

PyTorch 参考实现
导出精度验证基准

→ {op}_golden.py

4 方案设计

API 映射和计算流设计
Tiling 策略 和 Loop 结构

→ DESIGN.md

5 代码实现

Kernel 源码生成
编译调试与运行

→ {op}_impl.py

6 精度调试

前端写法检查
精度对比定位

→ 精度修复

7 性能调优

多轮迭代优化
性能对比输出

→ 调优报告

流程详解 - 阶段1~2

1 需求理解

orchestrator 执行

- 解析自然语言中的公式 / shape / dtype 约束
- 提取计算逻辑与约束，标注置信度
- 生成结构化 SPEC.md(含数据流图)

2 API 探索

orchestrator 执行

- 将复合运算拆解为原子操作
- 检索 PyPTO API 库，验证组合可行性
- 输出 API_REPORT.md(含 Tiling 建议)

产出物

产出物目录

```
ops/{op}/
├── SPEC.md        ← Stage 1
└── API_REPORT.md  ← Stage 2
```

SPEC.md

算子名称: operation_name
数学公式: formula
算法描述: algorithm
数据规格: shape/dtype/dynamic_axis
精度标准: atol/rtol
性能要求: us
参考信息: 外部参考实现

API_REPORT.md

公式分解: operation
API映射: expr 到 PyPTO API 映射
约束检查: API 约束
参考实现: models & examples 下的可参考的用例
可行性: 风险评估和是否可行

Stage 1~2 由 orchestrator 直接调用 pypto-intent-understand 和 pypto-api-explore Skills

流程详解 - 阶段3~4

3 Golden 生成

analyst 执行

- 基于 SPEC.md 生成纯 PyTorch 参考实现
- 导出 {op}_golden() 标准函数签名
- 生成精度验证基准数据集

4 方案设计

analyst 执行

- 分析算子数据流模式与计算特征
- 规划 Tiling 策略与 Loop 结构
- 设计验证方案，输出 DESIGN.md

产出物

产出物目录

```
ops/{op}/
├── SPEC.md
├── API_REPORT.md
├── {op}_golden.py      ← Stage 3
└── DESIGN.md           ← Stage 4
```

golden.py

```
def {op}_golden(x, scale):
    y = x * scale
    y = torch.round(y)
    return torch.clamp(y,qmin,qmax)/scale
```

DESIGN.md

功能描述: 算子功能描述
数学公式: 数学公式描述
API 映射：公式到 PyPTO API 的映射关系
实现设计：计算流设计、推荐配置、Tiling策略、Loop结构
验证方案：测试用例设计和精度标准
交付件清单: 交付件目录结构

Stage 3~4 由 pypto-op-analyst 在隔离上下文中执行，输出精度基准与设计方案

CANN

流程详解 - 阶段5~6

5 代码实现

developer 执行

- 根据 DESIGN.md 自动生成 Kernel 源码
- 首跑前预检: golden 可导入 + 设计完整
- 三态判定: PASS / PRECISION_FAIL / 运行失败

6 精度调试

developer 执行

- 仅在 PRECISION_FAIL 时触发进入
- 排查前端写法问题
- 逐层精度对比定位问题算子

产出物

产出物目录

```
ops/{op}/
├── SPEC.md
├── API_REPORT.md
├── {op}_golden.py
├── DESIGN.md
├── {op}_impl.py        ← Stage 5
└── test_{op}.py        ← Stage 5
```

{op}_impl.py 核心结构

```
@pypto.frontend.jit
def qat_forward(x, scale):
    # 1. view
    # 2. calc
    # 3. assemble
```

test_{op}.py

```
x = gen_random_data(shape)
y_npu = impl.{op}_forward(x, scale)
y_cpu = golden.{op}_forward(x, scale)
assert_allclose(y_npu, y_cpu, atol, rtol)
```

Stage 5~6 由 pypto-op-developer 执行，从代码生成到精度收敛全程自动化

CANN

流程详解 - 阶段7

7 性能调优

perf-tuner 执行

- 记录 Kernel 基线耗时、shape、精度指标
- 搜索 Tiling 参数、Loop 切分、流水配置
- 每轮: 分析→调优→复验→采纳/回滚
- 达到最大调优轮次或连续多轮无性能提升后终止

完整产出物

产出物目录

```
ops/{op}/
├── SPEC.md
├── API_REPORT.md
├── golden.py
├── DESIGN.md
├── IMPL.py
├── TEST_{OP}.py
├── PERF_REPORT.md   ← Stage 7
└── README.md
```

README.md

算子概述：算子名称和描述
算子公式：formula
算子规格：shape/dtype等
目录结构：算子目录结构
运行方式：cd custom/{op} && python test_{op}.py

PERF_REPORT.md

调优性能: 提升 5 倍
实际提升: 提升 5.9 倍
调优时长: 73 分钟
迭代轮次: 6
调优策略: Tiling、loop_unroll

Stage 7 由 pypto-op-perf-tuner 执行，精度通过后自动进入性能迭代优化

CANN

目录

Part 1 背景介绍

Part 2 Agent 架构

Part 3 案例实操

环境准备

环境要求

依赖	版本要求
昇腾硬件	Atlas A2 / A3 推理或训练系列
CANN	≥ 8.0
Python / PyTorch	≥ 3.9 / ≥ 2.6

支持平台

- OpenCode（默认）
- Claude Code

>_ CANNBot 部署

```
$ git clone https://gitcode.com/cann/skills.git
$ cd skills/teams/pypto-op-orchestrator
$ bash init.sh project opencode
```

>_ 打开 opencode

```
$ opencode
```


CANN

快速上手

示例1(交互式需求澄清)：“我想开发一个融合算子"

> 我想开发一个融合算子

Thinking: The user wants to develop a fusion operator. This is a PyPTO operator development request, and I need to follow the orchestrator workflow. However, the user's description is quite vague - "融合算子" (fusion operator) is very generic. I need to understand what specific fusion operator they want to develop.

Let me follow the startup flow:
1. Parse operator name and working directory
2. Check if .orchestrator_state.json exists
3. If not, initialize state

But first, I need to understand what the user actually wants. The term "融合算子" could mean many things - it could be a fusion of multiple operations, or a specific fusion operation. I should ask the user for more details before proceeding with Stage 1 (intent understanding).

Actually, looking at the orchestrator instructions, Stage 1 is "需求理解" (intent understanding) which uses the pypto-intent-understand skill. But the user hasn't given me enough information yet. Let me ask for more details about what kind of fusion operator they want to develop.

Wait, I should think about this more carefully. The user said "我想开发一个融合算子" which is quite vague. I need to ask them what specific operator they want - what operations to fuse, what the input/output tensors look like, etc.

Let me ask the user for clarification.

您好！您提到想开发一个"融合算子"，但这个描述比较宽泛。为了准确地启动开发流程，我需要了解更多细节。

请补充以下信息，帮助我理解您的需求：

1. 算子名称/类型：具体是哪种融合算子？例如：
   - LayerNorm + MatMul 融合
   - Softmax + MatMul 融合（如 Flash Attention 风格）
   - Bias + Add + Activation 融合（如 BiasAddGelu）
   - 其他特定操作的组合

2. 输入输出：涉及哪些 Tensor，各自的 dtype 和 shape 是怎样的？

3. 参考来源：是否有参考的 PyTorch API（如 F.scaled_dot_product_attention）、论文公式、或已有的算子规格？

4. 精度要求：是否需要与 PyTorch float32 结果做精度对比？

5. 性能目标：是否有特定的性能指标要求？

有了这些信息，我就可以启动 Stage 1（需求理解），为您生成结构化的 SPEC.md 并进入后续开发流程。

▣ Pypto-Op-Orchestrator · GLM-5-Turbo · 18.1s

https://gitcode.com/cann/skills

CANN

快速上手

示例2(常见算子开发)：“开发一个 softmax 算子"

```
开发一个softmax算子
```

```
Thinking: 用户要求开发一个 softmax 算子。作为 pypto-op-orchestrator，我需要按照启动流程来执行：

1. 解析算子名 {op} 与工作目录 custom/{op}/
2. 检查 custom/{op}/.orchestrator_state.json 是否存在
3. 若不存在状态文件：先通过 state_transition 初始化状态
4. 读取当前目录下已存在的工件
5. 从 current_stage 开始逐阶段推进

算子名是 softmax，工作目录是 custom/softmax/。

让我先检查目录和状态文件是否存在。
```

我来启动 **softmax** 算子的端到端开发流程。首先进行启动前的环境检查。

```
$ ls [path=/workspace/projects/code/pypto/custom]
$ ls /workspace/projects/code/pypto/custom/softmax/ 2>/dev/null || echo "DIR_NOT_EXISTS"
```

算子目录 **custom/softmax/** 不存在，这是新算子开发场景。现在创建目录并初始化状态机。

```
$ mkdir [path=/workspace/projects/code/pypto/custom/softmax]
$ state_transition [opDir=custom/softmax, action=start_stage, stage=1]
```

状态机已初始化，**Stage 1** 开始。现在加载 **pypto-intent-understand Skill** 执行需求理解。

```
# Todos

[•] Stage 1: 需求理解 - 调用 pypto-intent-understand 生成 SPEC.md
[ ] Stage 2: API 探索 - 调用 pypto-api-explore 生成 API_REPORT.md
[ ] Stage 3: Golden 生成 - 调度 pypto-op-analyst 生成 softmax_golden.py
[ ] Stage 4: Design 设计 - 调度 pypto-op-analyst 生成 DESIGN.md
[ ] Stage 5: 代码实现 - 调度 pypto-op-developer 生成 impl/test
[ ] Stage 6: 精度修复 (若需要)
[ ] Stage 7: 性能调优
```

```
→ Skill "pypto-intent-understand"
```

快速上手

示例3(自定义算子开发)：“读取 requirement.md，根据需求开发自定义算子"

读取 requirement.txt，根据需求开发自定义算子（开发前打印原始需求）

Thinking: The user wants me to read a requirement.txt file and then develop a custom operator based on the requirements. Let me first find and read the requirement.txt file.

```
✱ Glob "**/requirement*.txt"
✱ Glob "**/requirement.txt"
✱ Glob "**/Requirement*.txt"
$ find /workspace/projects/code/pypto -maxdepth 3 -iname "requirement*" -o -iname "需求*" 2>/dev/null | head -20
```

找到了 requirement.md，让我读取它。

~ Reading file...

好的，以下是原始需求：

📋 原始需求

算子名称： custom_add_norm

公式： $y = \frac{(x_1 + x_2) - \mu}{\sqrt{\sigma^2 + \epsilon}} \cdot \gamma + \beta$

- μ 和 σ^2 分别是 (x_1+x_2) 在最后一个维度上的均值和方差（沿 hidden_size 维度）
- $\epsilon = 1e-5$
- γ 和 β 为可学习参数，形状与最后一个维度一致

规格：

类型	shape	dtype
输入 x1	[*, hidden_size]	bfloat16
输入 x2	[*, hidden_size]	bfloat16
输入 gamma	[hidden_size]	bfloat16
输入 beta	[hidden_size]	bfloat16
输出 y	[*, hidden_size]	bfloat16

精度标准： atol=0.0001, rtol=0.0078125

特殊要求： batch 维度（* 部分）是动态轴

现在开始按编排流程启动算子开发。首先检查状态文件和现有工件。

CANN

CANNBot PyPTO — 让融合算子开发变得简单

CANNBot PyPTO通过Agent驱动的方式，
将复杂的算子开发流程化繁为简。
从需求描述到算子运行，全流程自动化：
设计方案、编写代码、调试调优。

现在，开始您的CANNBot PyPTO之旅！

欢迎关注、使用、贡献，共建社区！

https://gitcode.com/cann/pypto &

https://gitcode.com/cann/skills

群聊：CANNBot与Skills开源交流群

该二维码7天内(4月21日前)有效，重新进入将更新

扫码加入微信交流群

CANN

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

上CANN社区获取干货

关注CANN公众号获取资讯

https://gitcode.com/cann/skills